

Apache Beam/Python: modelo de programación

Streaming de datos y sus aplicaciones

Maestría en Inteligencia Artificial

Facultad Politécnica - Universidad Nacional de Asunción

Rodrigo Parra, M.Sc.

Clase 5

Contents

1. Del diseño al código

- La clase anterior definió el significado; hoy lo hacemos ejecutable
- El reto no es escribir un bucle sobre una lista
- Hoy construiremos un pipeline de ventas
- Resultados de aprendizaje
- Cuatro preguntas guían la implementación

2. Cuatro piezas del modelo

- Beam separa la lógica de la plataforma de ejecución

- Pipeline es el grafo que estamos construyendo
- PCollection representa datos dentro del pipeline
- PCollection no es una lista de Python
- PTransform convierte colecciones en colecciones
- El runner decide cómo ejecutar el grafo
- Batch y streaming comparten el mismo vocabulario

3. Leer el código como un grafo

- Un pipeline mínimo cabe en pocas líneas
- El operador pipe aplica una transformación
- Las etiquetas facilitan el diagnóstico del grafo
- Create convierte datos pequeños en una PCollection

- Cada asignación nombra una etapa del grafo
- Ramificar no copia la colección conceptualmente
- El contexto with hace explícito el ciclo de vida
- Actividad: traduce código a grafo

4. Transformar en paralelo

- Map produce exactamente una salida por entrada
- FlatMap puede emitir cero, una o muchas salidas
- Filter conserva solo los elementos que cumplen una condición
- ParDo expresa lógica más general
- ParDo permite separar salidas válidas y rechazadas
- Las funciones deben tolerar paralelismo y reintentos

- setup y teardown administran recursos por instancia
- Actividad: elige Map, FlatMap, Filter o ParDo

5. Reunir lo que pertenece junto

- Agrupar requiere pares clave–valor
- GroupByKey reúne todos los valores de cada clave
- GroupByKey expone valores que quizá no necesitas
- CombinePerKey habilita agregación parcial
- Los combiners frecuentes expresan intención
- En streaming, toda agregación vive en una ventana

6. Del archivo al resultado

- Los conectores de I/O también son transformaciones

- ReadFromText emite una línea por elemento
- WriteToText puede producir varios shards
- PipelineOptions separa código y configuración
- DirectRunner es el primer entorno de ejecución

7. Laboratorio interactivo

- La práctica continúa en un notebook Marimo
- Fuentes y atribuciones

Del diseño al código

Clase 4

- tiempo de evento
- ventanas y watermarks
- triggers y lateness
- acumulación de resultados

Clase 5

- grafo de transformaciones
- colecciones distribuidas
- funciones paralelas
- ejecución y pruebas

Idea clave

Apache Beam convierte decisiones de procesamiento en un programa portable que un runner puede ejecutar.

Python local

```
result = []  
for event in events:  
    if event["valid"]:  
        result.append(normalize(event))
```

Ejecución distribuida

- los datos pueden no caber en memoria
- el orden global no está garantizado
- varias máquinas ejecutan en paralelo
- un elemento puede reprocesarse
- la fuente puede no terminar

Beam pide describir **qué transformación** aplicar; el runner decide cómo distribuirla.

Entrada

Eventos JSON con event_id, store_id, event_time, amount y status.

Leer

líneas JSON



Transformar

validar y agregar



Escribir

totales por tienda

Pregunta analítica

¿Cuál es el monto confirmado por tienda?

Al final de la clase deberías poder

- Explicar Pipeline, PCollection, PTransform y runner.
- Encadenar Create, Map, Filter, ParDo y transformaciones por clave.
- Elegir entre GroupByKey y combiners.
- Leer y escribir datos con conectores de I/O.
- Ejecutar localmente con DirectRunner.
- Probar transformaciones con TestPipeline y `assert_that`.
- Organizar un proyecto mínimo reproducible.

Datos

¿Qué representa cada elemento y cuál es su tipo?

Transformación

¿Qué operación produce la siguiente colección?

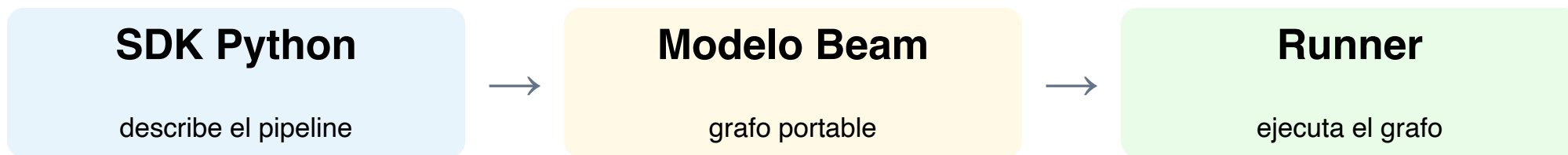
Ejecución

¿Qué runner y opciones materializan el grafo?

Evidencia

¿Qué prueba demuestra que el resultado es correcto?

Cuatro piezas del modelo



Consecuencia

La misma intención puede ejecutarse localmente o en un motor distribuido compatible, sujeto a las capacidades del runner.

Durante la definición

- registra transformaciones
- conecta entradas y salidas
- conserva opciones de ejecución
- valida parte de la estructura

Durante la ejecución

El runner traduce el grafo a tareas, etapas, workers, shuffles y operaciones de I/O.

Idea clave

Construir el pipeline no equivale a recorrer inmediatamente todos los datos.

Distribuida

Puede abarcar muchas máquinas y particiones.

Inmutable

Una transformación produce otra PCollection.

Acotada o no acotada

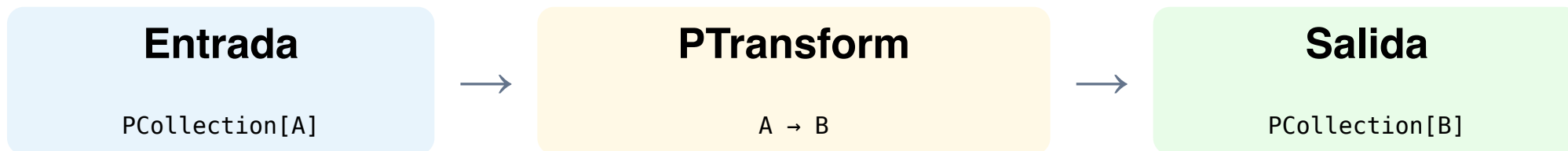
Puede representar un dataset finito o un stream continuo.

Sin acceso aleatorio

No se consulta como una lista local por índice.

Fuente: [Apache Beam Programming Guide — PCollections](#).

Propiedad	Lista local	PCollection
Ubicación	memoria del proceso	runner / sistema distribuido
Tamaño	conocido al observarla	puede ser enorme o infinito
Orden	posición explícita	no asumir orden global
Mutación	posible	lógicamente inmutable
Inspección	<code>items[0]</code> , <code>len(items)</code>	mediante transforms y sinks



Ejemplos

Map, Filter, ParDo, GroupByKey, CombinePerKey, ReadFromText y WriteToText son transformaciones.

DirectRunner

- desarrollo local
- validación funcional
- pruebas pequeñas
- no optimizado para producción

Runners distribuidos

- paralelismo real
- estado y checkpoints
- escalado y operación
- capacidades específicas

Idea clave

Portabilidad no significa que todos los runners tengan exactamente las mismas capacidades operativas.

Lo que permanece

PCollection + PTransform + runner + conectores de I/O.

Fuente acotada

Archivo, tabla o colección finita:
el trabajo puede terminar.

Fuente no acotada

Kafka o una suscripción: el
trabajo continúa y las
agregaciones necesitan
ventanas.

La unificación está en el modelo; las propiedades de la fuente siguen importando.

Leer el código como un grafo

main.py

```
import apache_beam as beam

with beam.Pipeline() as pipeline:
    (
        pipeline
        | "Create words" >> beam.Create(["beam", "python", "beam"])
        | "Uppercase" >> beam.Map(str.upper)
        | "Write output" >> beam.io.WriteToText("output/words")
    )
```

Al salir del bloque with, Beam ejecuta el pipeline y espera su finalización.

Lectura de izquierda a derecha

```
entrada | "Etiqueta" >> Transformación
```

Entrada

Pipeline O PCollection

Etiqueta

Nombre legible y único en el grafo.

Transformación

Produce cero o más salidas.

Idea clave

La sintaxis parece una tubería, pero el resultado es un grafo de ejecución.

Débil

```
rows | beam.Map(parse)
```

¿Qué Map falló?

Informativo

```
rows | "Parse sale JSON" >>  
beam.Map(parse)
```

El nombre expresa intención.

Las etiquetas aparecen en visualizaciones, logs, métricas y mensajes de error.

Útil para aprender y probar

```
events = pipeline | "Seed events" >> beam.Create([
    {"store_id": "A", "amount": 120_000, "status": "CONFIRMED"},
    {"store_id": "B", "amount": 80_000, "status": "PENDING"},
    {"store_id": "A", "amount": 50_000, "status": "CONFIRMED"},
])
```


No es una fuente de producción

Para grandes volúmenes, usar conectores que lean desde almacenamiento o sistemas de mensajería.

Pipeline explícito

```
raw = pipeline | "Read sales" >> beam.io.ReadFromText(input_path)
parsed = raw | "Parse JSON" >> beam.Map(parse_sale)
valid = parsed | "Keep valid" >> beam.Filter(is_valid)
keyed = valid | "Key by store" >> beam.Map(
    lambda event: (event["store_id"], event["amount"])
)
totals = keyed | "Sum per store" >> beam.CombinePerKey(sum)
```

Nombres intermedios claros facilitan pruebas, revisión y reutilización.

Eventos válidos

PCollection[event]



Totales

agregación

Auditoría

muestreo

Modelo

Una `PCollection` puede alimentar varias transformaciones; el runner optimiza la ejecución.

Con with

```
with beam.Pipeline() as p:  
    output = ...
```

Ejecuta al salir del bloque.

Control manual

```
p = beam.Pipeline()  
output = ...  
result = p.run()  
result.wait_until_finish()
```

Para las primeras prácticas usaremos el bloque with.

Pipeline

```
approved = (  
  p  
  | beam.Create(events)  
  | beam.Filter(lambda e: e["status"] == "APPROVED")  
  | beam.Map(lambda e: (e["country"], 1))  
  | beam.CombinePerKey(sum)  
)
```

En parejas

Identifiquen entradas, salidas, tipo aproximado de cada elemento y punto donde ocurre el shuffle.



10 minutos

Al volver: transformaciones elemento a elemento, claves y agregaciones.

Transformar en paralelo

Lista mental

$A \rightarrow B$

[1, 2, 3, 4]

$x \rightarrow x * 10$

[10, 20, 30, 40]

En Beam

```
tens = numbers | beam.Map(  
    lambda x: x * 10  
)
```

4 elementos entran; 4 elementos salen.

Buenos usos

Normalizar campos, proyectar atributos, crear pares clave–valor o formatear salidas.

Lista mental

$A \rightarrow \text{Iterable}[B]$

```
["beam python", "datos en  
flujo"]
```

$\text{line} \rightarrow \text{line.split}()$

```
["beam", "python", "datos",  
"en", "flujo"]
```

En Beam

```
words = lines |  
  beam.FlatMap(  
    lambda line:  
      line.split()  
  )
```

Idea clave

Si una función retorna una lista dentro de Map, esa lista es un solo elemento; FlatMap emite sus componentes.

Lista mental

[1, 2, 3, 4, 5, 6]

$x \rightarrow x \% 2 == 0$

[2, 4, 6]

En Beam

```
evens = numbers |  
    beam.Filter(  
        lambda x: x % 2 == 0  
    )
```

Idea clave

True: el mismo elemento continúa. False: no aparece en la salida.

DoFn

```
class ParseSale(beam.DoFn):  
    def process(self, line):  
        try:  
            event = json.loads(line)  
            yield event  
        except json.JSONDecodeError:  
            return  
  
events = lines | beam.ParDo(ParseSale())
```

process puede emitir cero, uno o varios elementos mediante yield.

Validación con salida etiquetada

```
class ValidateSale(beam.DoFn):  
    def process(self, event):  
        if event.get("store_id") and event.get("amount", 0) >= 0:  
            yield event  
        else:  
            yield beam.pvalue.TaggedOutput("invalid", event)  
  
result = events | beam.ParDo(ValidateSale()).with_outputs(  
    "invalid", main="valid"  
)  
valid = result.valid  
invalid = result.invalid
```

Los datos rechazados se vuelven observables en lugar de desaparecer silenciosamente.

Preferir

- funciones deterministas
- entradas y salidas explícitas
- configuración inmutable
- efectos externos idempotentes

Evitar

- mutar variables globales
- depender del orden
- abrir una conexión por elemento
- usar `print` como resultado

Idea clave

El runner puede crear varias instancias de la función y reprocesar elementos.

Patrón conceptual

```
class Enrich(beam.DoFn):  
    def setup(self):  
        self.client = build_client()  
  
    def process(self, event):  
        yield {**event, "segment": self.client.lookup(event["user_id"])}  
  
    def teardown(self):  
        self.client.close()
```

Cuidado

Estos métodos reducen inicializaciones costosas, pero no convierten un servicio externo en transaccional.

Necesidad	Elección	Por qué
Extraer amount	¿?	una salida
Separar palabras	¿?	muchas salidas
Retener confirmadas	¿?	predicado
Validar y desviar errores	¿?	salidas etiquetadas

Reunir lo que pertenece junto

Antes de agregar

```
keyed_amounts = confirmed | "Key amount by store" >> beam.Map(  
    lambda event: (event["store_id"], event["amount"])  
)
```


Clave

Decide qué elementos deben encontrarse.

Valor

Contiene lo que será agregado.

Entrada

(A, 120)
(B, 80)
(A, 50)



Salida

(A, [120, 50])
(B, [80])

Costo

El runner debe redistribuir datos por clave: aparece un **shuffle**.

Patrón costoso

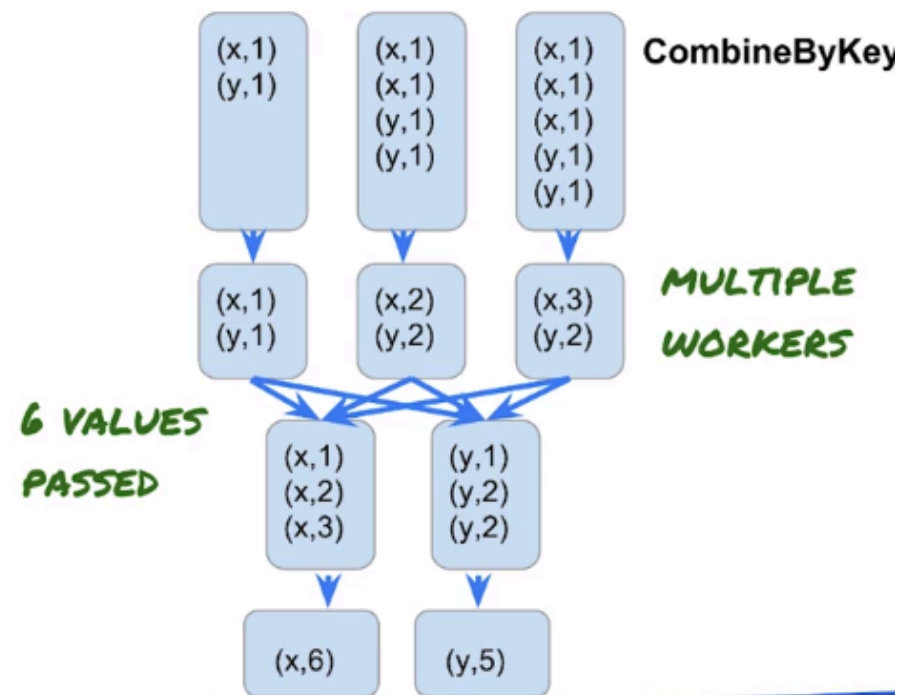
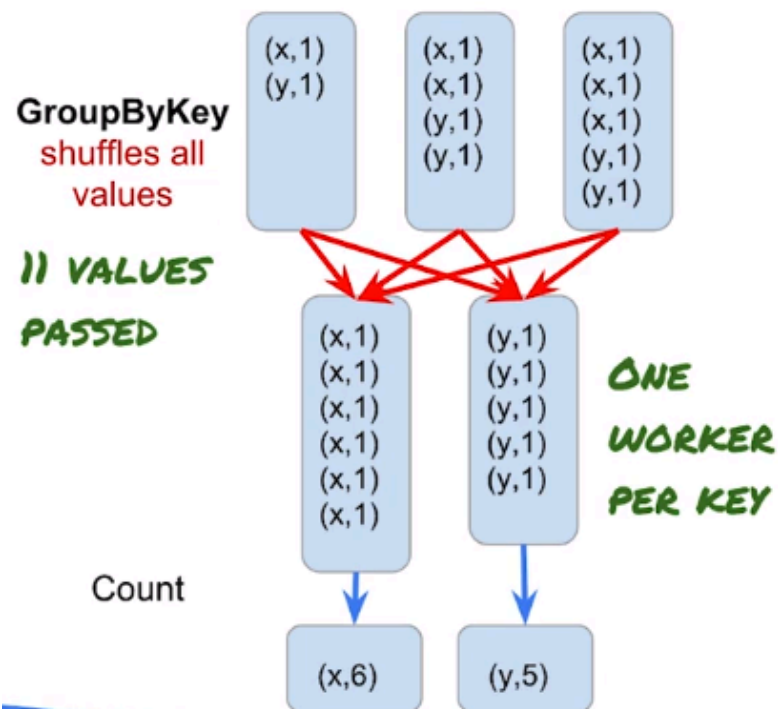
```
grouped = pairs |  
beam.GroupByKey()  
totals = grouped |  
beam.Map(  
    lambda kv: (kv[0],  
sum(kv[1]))  
)
```

Patrón declarativo

```
totals = pairs |  
beam.CombinePerKey(sum)
```

Idea clave

Para suma, conteo o promedio, expresa la agregación con un combiner.



Idea clave

GroupByKey mueve valores individuales; CombinePerKey puede resumirlos en cada worker antes del **shuffle**.

Transformación	Pregunta	Ejemplo
<code>Count.PerKey()</code>	¿cuántos por clave?	eventos por tienda
<code>CombinePerKey(sum)</code>	¿cuánto por clave?	monto confirmado
<code>Mean.PerKey()</code>	¿cuál es el promedio?	ticket medio
<code>Top.PerKey(n)</code>	¿cuáles son los mayores?	top transacciones
<code>CombineGlobally(...)</code>	¿cuál es el total global?	monto total

Con fuente acotada

La ventana global puede producir un resultado al finalizar el input.

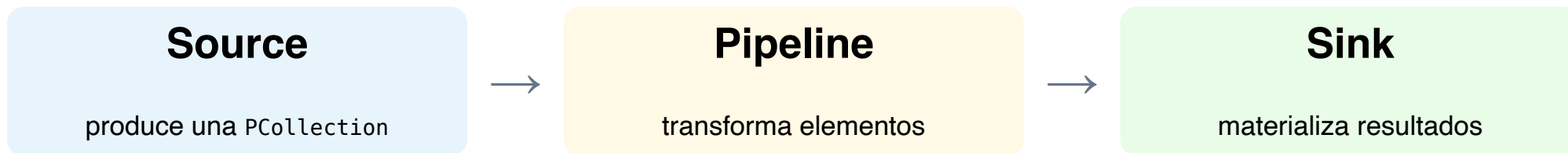
Con fuente no acotada

Una agregación global no tiene fin natural: necesitamos ventanas y una política de emisión.

Idea clave

La clase 4 definió el contrato temporal; en la clase 6 lo implementaremos con las APIs de windowing de Beam.

Del archivo al resultado



Ejemplos

Archivos, bases de datos, objetos, colas y logs se integran mediante conectores.

Lectura

```
lines = pipeline | "Read input" >> beam.io.ReadFromText(  
    options.input  
)
```

Separación de responsabilidades

El conector lee bytes y líneas; una transformación posterior interpreta JSON, CSV o el esquema del dominio.

Escritura

```
formatted | "Write totals" >> beam.io.WriteToText(  
    options.output,  
    file_name_suffix=".jsonl"  
)
```

Resultado posible

`totals-00000-of-00003.jsonl`

Motivo

Workers escriben particiones en paralelo.

Opciones propias

```
from apache_beam.options.pipeline_options import PipelineOptions

class SalesOptions(PipelineOptions):
    @classmethod
    def _add_argparse_args(cls, parser):
        parser.add_argument("--input", required=True)
        parser.add_argument("--output", required=True)

options = SalesOptions()
with beam.Pipeline(options=options) as pipeline:
    ...
```

Rutas, runner y parámetros cambian sin editar la lógica del pipeline.

Comando

```
python -m sales_pipeline \  
  --input data/sales.jsonl \  
  --output output/totals \  
  --runner DirectRunner
```

Su propósito

Ejecutar localmente, detectar incompatibilidades y obtener retroalimentación rápida.

Apache Beam indica que DirectRunner está orientado a prueba y desarrollo, no a eficiencia de producción.

Laboratorio interactivo

Incluye

- ejemplos interactivos con listas
- pipeline completo de ventas
- distribución de claves
- comparación de tráfico
- pruebas automatizadas

Repositorio público

github.com/rparrapy/streaming-fpuna-clase5

Notebook Python + Docker +
datos + src/ + pruebas.

Ejecutar

```
git clone https://github.com/rparrapy/streaming-fpuna-clase5.git  
cd streaming-fpuna-clase5  
docker compose up --build
```

- Modelo, transformaciones, claves y combiners: [Apache Beam Programming Guide](#) y [Python transform catalog](#).
- Agregación parcial: [Apache Beam — CombinePerKey](#).
- Ejecución local: [Apache Beam — Direct Runner](#).
- Imagen comparativa GroupByKey/combine: [Juan Ignacio Sánchez Lara](#).
- Notebook reproducible: [Marimo documentation](#) y [Deploying Marimo with Docker](#).

Material de clase

El caso de ventas, sus datos y el proyecto Docker fueron elaborados para esta práctica.

¿Preguntas?

 rmparra@pol.una.py

 Laboratorio: Marimo + Apache Beam + Docker

 Próxima clase: ventanas, estado y efectos externos